Reproduction of Two RAG Baselines: Social-Web Indirect Prompt Injection and RAG-based Factual Accuracy

Assignment 02 — Reproduction of Selected Research Paper Student: Mian Tayyab ur Rehman (220) **Program:** Deep Learning — Department of Computer Science, Bahria University, Islamabad (H-11 Campus) **Date:** 9 November 2026

Abstract

This report reproduces the two baseline systems approved in Assignment 01. **Baseline 1** — OpenRAG-Soc (Guo & Wei, 2026) — measures how often a Retrieval-Augmented Generation (RAG) system obeys instructions hidden in retrieved web content (indirect prompt injection), and how three defenses reduce the Attack-Success Rate (ASR). **Baseline 2** — the CMU RAG pipeline (Li, Yuan & Zhang, 2024) — improves factual accuracy with a retriever + reranker + Llama-2 generator and reports an ablation over Recall, F1, cosine and BLEU. Both were re-implemented in PyTorch/HuggingFace and run on a single Colab GPU using 4-bit quantization. The core findings of both papers reproduced in direction and ordering; absolute numbers differ for documented reasons (compute, missing data, under-specified details).

1. Introduction and Research Objective

RAG grounds an LLM's answers in retrieved documents, which reduces hallucination (Baseline 2) but also lets hidden instructions in retrieved pages hijack the model (Baseline 1). The objective of this assignment is to **implement and reproduce both baselines** — rebuild each data pipeline, re-implement each model/harness, run each paper's evaluation, and compare the reproduced results against the originals — so they form a reliable foundation for the Assignment 03 improvement (an intent-based query guardrail).

2. Dataset and Preprocessing

Baseline 2 (CMU knowledge base). Reused from the authors' release (`github.com/anlp-team/LTI_Neural_Navigator`, MIT). The retrieval corpus is CMU/LTI web pages and papers; the QA pairs are in `embedder_dataset/*.json` (key `qa_list`). The release contains **16,329 QA pairs** (a subset of the paper's 34,781). We chunk the corpus and make a random **80/20 split** (seed 42 → ~13,064 train / ~3,265 test). Deviation: the paper uses 1000-word chunks, but that overflows Llama-2's 4,096-token context when 5 passages are retrieved, so we chunk at **250 words**.

Baseline 1 (social-web corpus). The paper does not release its corpus, so we **regenerate it synthetically** from the published recipe: social-style pages across five carriers (hidden spans, off-screen CSS, alt text, ARIA, zero-width). Each page states one fact about a fictional entity (so utility is measurable) and each injected payload carries a unique canary token (so obedience is detectable). We ran a regenerated corpus of **504 pages (~550 payload queries)**. Deviation:

zero-width pages place the payload beside zero-width marker characters rather than encoding it in invisible characters.

3. Baseline Model Architecture

Baseline 2 pipeline: `mxbai-embed-large-v1` embeddings → MMR top-10 retrieval → `bge-reranker-large` keeps top-5 → **Llama-2-7b-chat** generates (prompt from the paper's Appendix B.2).

Baseline 1 harness: page → ingest (optional sanitize / normalize) → chunk (512/128) → **BM25** retrieval (k=5) → prompt (standard / guarded / attribution-gated) → **Llama-3-8B-Instruct** → answer → ASR detector. The four reported configurations are **Vanilla**, **Sanitized** (strip hidden HTML), **Normalized** (NFKC Unicode), and **All Defenses** (sanitize + normalize + attribution-gating).

4. Experimental Setup

- **Environment:** Google Colab, NVIDIA A100 (40 GB), CUDA 12.x, PyTorch 2.x, `transformers ≥ 4.44`; generators loaded in 4-bit NF4 (bitsandbytes). Exact versions in `results/environment.json`.
- **Baseline 2 core fine-tune:** Llama-2-7b-chat, QLoRA, LoRA r=16, α=32, lr 2e-4, batch 8, INT4 (paper's Sec. 5.1 settings); run at `max_steps=100` for the time budget.
- **Baseline 2 embedding fine-tune:** attempted with `MultipleNegativesRankingLoss` but **not used** — it collapsed (see §6).
- **Evaluation:** Baseline 2 — mean over 2 draws × 64 test QA pairs (paper uses 4×128); test split held out. Baseline 1 — ASR per carrier over ~550 queries, four defense configs, seed 0.

5. Results and Comparison with the Original Paper

Difference = Reproduced − Original.

5.1 Baseline 2 — RAG factual-accuracy ablation

Configuration	Metric	Original	Reproduced	Difference
Baseline (w/o RAG)	Recall	0.361	0.320	−0.041
	F1	0.186	0.195	+0.009
Raw RAG Pipeline	Recall	0.409	0.369	−0.040
	F1	0.289	0.176	−0.113
+ Fine-tuned Embedding	F1	0.304	not reproduced	—
+ Fine-tuned Core	Recall	0.448	0.380	−0.068
	F1	0.211	0.162	−0.049

Reproduced: RAG improves recall (0.320 → 0.369 → 0.380), and core fine-tuning raises recall while lowering F1 (0.176 → 0.162) — the paper's key trade-off. **Not reproduced:** raw RAG's F1 did not exceed no-RAG as in the paper, because RAG answers are longer (higher recall, lower precision) and token-overlap metrics penalise the extra words; absolute scores are lower throughout (4-bit, data subset, `max_steps=100`, small sample).

5.2 Baseline 1 — Indirect prompt injection (macro ASR %)

Defense	Original	Reproduced	Difference
Vanilla	24.9	69.2	+44.3
Sanitized	13.1	25.9	+12.8
Normalized	21.3	68.8	+47.5
All Defenses	4.7	0.2	−4.5

Per-carrier Vanilla ASR (reproduced): hidden spans 76.4, off-screen 65.7, alt text 65.7, ARIA 65.4, zero-width 72.6.

Reproduced: the defense ordering **Vanilla ≈ Normalized > Sanitized > All Defenses** matches the paper; sanitization sharply cuts the HTML carriers; All Defenses nearly eliminates obedience (69.2 → 0.2). **Supporting results:** retrieval poisoning degrades ranking under Vanilla (ΔMRR@10 0 → −0.45 at 10% poison) while Sanitized+Normalized prevents it (Figure 1); the ASR detector is canary-exact (P=R=F1=1.0). **Differs:** absolute ASR is ~3× the paper's (synthetic corpus, blatant payloads, exact detector, different generator), and zero-width ASR does not drop under Normalized (our carrier simplification).

6. Reproducibility Challenges and Discussion

Was the baseline implemented successfully? Yes — both pipelines run end-to-end and produce the papers' metrics with the expected qualitative patterns.

How do the results compare? Direction and ordering reproduced for both; absolute magnitudes differ (Baseline 2 lower, Baseline 1 higher) for the documented reasons.

Which details were hard to reproduce or missing? Baseline 1's corpus is **not released** (we regenerated it); Baseline 1's exact detector and "coverage-adjusted ASR" are under-specified; Baseline 2's Appendix D defines P/R/F1 generically (we use token-overlap); the released QA set is a subset (16,329 of 34,781).

Did hardware/config affect results? Yes: 4-bit quantization, the regenerated corpus, smaller sample/step counts, and the 250-word chunking all shift absolute values while preserving the findings.

Main challenges encountered. (1) **Embedding fine-tune collapse** — `MultipleNegativesRankingLoss` drove the training loss to 0 within one epoch and held-out retrieval recall fell from 0.785 to 0.0, because the ~16k QA answers are narrow and templated; we therefore used the pretrained embedding. (2) **Context overflow** — 1000-word chunks × 5 passages overflowed the generator; fixed by chunking at 250 words. (3) Fitting a 7–8B generator and fine-tuning on one GPU required 4-bit quantization.

How this supports Assignment 03. Baseline 2 provides the clean factual-accuracy pipeline to defend, and Baseline 1 provides the ASR harness and defense framework. The proposed query guardrail can be evaluated with the same ASR/utility/latency metrics by adding a direct-attack query set and a pre-generation filter.

7. Conclusion and Next Steps

Both RAG baselines were re-implemented and reproduced under a single-GPU budget. The reproductions recover the papers' core findings — RAG and retrieval improve factual coverage, and layered hygiene plus attribution-gating sharply reduce indirect-injection ASR — with all deviations documented. Next (Assignment 03): add direct-query attacks and an intent-based query guardrail, and measure the reduction in attack success while tracking the factual-accuracy baseline and the guardrail's latency.

References

1. Haoze Guo and Ziqi Wei. 2026. *Hidden-in-Plain-Text: A Benchmark for Social-Web Indirect Prompt Injection in RAG.* Proc. ACM Web Conference 2026 (WWW '26). DOI: 10.1145/3774904.3792853. arXiv:2601.10923.
2. Jiarui Li, Ye Yuan, Zehua Zhang. 2024. *Enhancing LLM Factual Accuracy with RAG to Counter Hallucinations: A Case Study on Domain-Specific Queries in Private Knowledge-Bases.* arXiv:2403.10446. Code/data: github.com/anlp-team/LTI_Neural_Navigator.
3. Touvron et al. 2023. *Llama 2: Open Foundation and Fine-Tuned Chat Models.*
4. Lee et al. 2024. *Open Source Strikes Bread — mxbai-embed-large-v1.*
5. Robertson and Zaragoza. 2009. *The Probabilistic Relevance Framework: BM25 and Beyond.*